Universidade de Brasília - UnB
Faculdade UnB Gama - FGA
Engenharia de Software

ARENA: plataforma robótica móvel com ROS 2 para mapeamento, navegação autônoma e comparação de algoritmos de controle

Autor: Felipe das Neves Freire
Orientador: Prof. Dr. Renato Coral Sampaio

Brasília, DF
2026



Felipe das Neves Freire

ARENA: plataforma robótica móvel com ROS 2 para mapeamento, navegação autônoma e comparação de algoritmos de controle

Monografia submetida ao curso de graduação em Engenharia de Software da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia de Software.

Universidade de Brasília - UnB

Faculdade UnB Gama - FGA

Orientador: Prof. Dr. Renato Coral Sampaio

Coorientador: (Titulação Acadêmica e Nome do Orientador)

Brasília, DF

2026

Felipe das Neves Freire

ARENA: plataforma robótica móvel com ROS 2 para mapeamento, navegação autônoma e comparação de algoritmos de controle/ Felipe das Neves Freire. – Brasília, DF, 2026-

53 p. : il. (algumas color.) ; 30 cm.

Orientador: Prof. Dr. Renato Coral Sampaio

Trabalho de Conclusão de Curso – Universidade de Brasília - UnB
Faculdade UnB Gama - FGA , 2026.

1. Palavra-chave01. 2. Palavra-chave02. I. Prof. Dr. Renato Coral Sampaio. II. Universidade de Brasília. III. Faculdade UnB Gama. IV. ARENA: plataforma robótica móvel com ROS 2 para mapeamento, navegação autônoma e comparação de algoritmos de controle

CDU 02:141:005.6

Felipe das Neves Freire

ARENA: plataforma robótica móvel com ROS 2 para mapeamento, navegação autônoma e comparação de algoritmos de controle

Monografia submetida ao curso de graduação em Engenharia de Software da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia de Software.

Trabalho aprovado. Brasília, DF, x de dezembro de 2026:

Prof. Dr. Renato Coral Sampaio
Orientador

Titulação e Nome do Professor
Convidado 01
Convidado 1

Titulação e Nome do Professor
Convidado 02
Convidado 2

Brasília, DF
2026

Agradecimentos

A inclusão desta seção de agradecimentos é opcional, portanto, sua inclusão fica a critério do(s) autor(es), que caso deseje(em) fazê-lo deverá(ão) utilizar este espaço, seguindo a formatação de *espaço simples e fonte padrão do texto (arial ou times, tamanho 12 sem negritos, aspas ou itálico.*

Caso não deseje utilizar os agradecimentos, deixar toda este arquivo em branco.

A epígrafe é opcional. Caso não deseje uma, deixe todo este arquivo em branco.

"Não vos amoldeis às estruturas deste mundo,
mas transformai-vos pela renovação da mente,
a fim de distinguir qual é a vontade de Deus:
o que é bom, o que Lhe é agradável, o que é perfeito.
(Bíblia Sagrada, Romanos 12, 2)

Resumo

O resumo deve ressaltar o objetivo, o método, os resultados e as conclusões do documento. A ordem e a extensão destes itens dependem do tipo de resumo (informativo ou indicativo) e do tratamento que cada item recebe no documento original. O resumo deve ser precedido da referência do documento, com exceção do resumo inserido no próprio documento. (. . .) As palavras-chave devem figurar logo abaixo do resumo, antecedidas da expressão Palavras-chave:, separadas entre si por ponto e finalizadas também por ponto. O texto pode conter no mínimo 150 e no máximo 500 palavras, é aconselhável que sejam utilizadas 200 palavras. E não se separa o texto do resumo em parágrafos.

Palavras-chaves: latex. abntex. editoração de texto.

Abstract

This is the english abstract.

Key-words: latex. abntex. text editoration.

Lista de ilustrações

Lista de tabelas

Tabela 1 – Componentes de hardware do robô 27
Tabela 2 – Ligações no Raspberry Pi 3 (numeração BCM e pino físico) 32
Tabela 3 – Principais tópicos do sistema 38

Lista de abreviaturas e siglas

4WD	*Four-Wheel Drive* – tração nas quatro rodas
ARM	*Advanced RISC Machines*
BCM	Broadcom (numeração dos pinos pelo processador)
BMS	*Battery Management System* – circuito de proteção da bateria
CC	Corrente contínua
CPR	*Counts Per Revolution* – contagens por volta
DDS	*Data Distribution Service*
DMA	*Direct Memory Access* – acesso direto à memória
EKF	*Extended Kalman Filter* – filtro de Kalman estendido
FGA	Faculdade UnB Gama
GPIO	*General Purpose Input/Output* – entrada e saída de propósito geral
I^2C	*Inter-Integrated Circuit*
IAE	*Integral of Absolute Error* – integral do erro absoluto
IMU	*Inertial Measurement Unit* – unidade de medição inercial
LiDAR	*Light Detection and Ranging*
LiPo	Polímero de lítio
LQR	*Linear Quadratic Regulator* – regulador linear quadrático
MJPEG	*Motion JPEG*
MOSFET	*Metal-Oxide-Semiconductor Field-Effect Transistor*
MPC	*Model Predictive Control* – controle preditivo baseado em modelo
PID	Proporcional, integral e derivativo
PPR	*Pulses Per Revolution* – pulsos por volta
PSO	*Particle Swarm Optimization* – otimização por enxame de partículas

PWM	*Pulse Width Modulation* – modulação por largura de pulso
RAM	*Random Access Memory* – memória de acesso aleatório
RGB	*Red, Green, Blue* – vermelho, verde e azul
ROS	*Robot Operating System*
RX	Recepção (UART)
SDK	*Software Development Kit* – kit de desenvolvimento de software
SLAM	*Simultaneous Localization and Mapping* – localização e mapeamento simultâneos
SSH	*Secure Shell*
TF	*Transform* – árvore de transformações do ROS
TX	Transmissão (UART)
UART	*Universal Asynchronous Receiver-Transmitter*
UnB	Universidade de Brasília
URDF	*Unified Robot Description Format*
USB	*Universal Serial Bus*
UVC	*USB Video Class*

Lista de símbolos

v_{max}	Velocidade linear teórica máxima do robô [m/s]
D	Diâmetro da roda [m]
n	Velocidade do eixo de saída do motor [rpm]
N_{CPR}	Contagens do encoder por volta da roda
R	Relação de redução da caixa do motor
ΔN	Contagens do encoder em um intervalo
Δs	Distância percorrida pela roda em um intervalo [m]
v	Velocidade linear do robô [m/s]
ω	Velocidade angular do robô [rad/s]
$v_e,\ v_d$	Velocidades das rodas esquerda e direita [m/s]
L	Distância entre os centros das rodas esquerda e direita [m]
$u(t)$	Sinal de controle (ciclo de trabalho)
$e(t)$	Erro de velocidade
K_p	Ganho proporcional do PID
K_i	Ganho integral do PID
K_d	Ganho derivativo do PID
$G(s)$	Função de transferência de um lado do robô
s	Variável de Laplace
$\Omega(s)$	Velocidade da roda no domínio de Laplace
$U(s)$	Ciclo de trabalho no domínio de Laplace
K	Ganho em regime da planta
τ	Constante de tempo da planta [s]
N	Horizonte de predição do MPC [passos]

r_k	Velocidade de referência no passo k
y_k	Velocidade da roda prevista pelo modelo no passo k
u_k	Ciclo de trabalho no passo k
Δu_k	Variação do ciclo de trabalho entre passos
λ	Peso do esforço de controle no custo do MPC
$\mathbf{x}_i$	Posição da partícula i no PSO (conjunto de ganhos)
$\mathbf{v}_i$	Velocidade da partícula i no PSO
$\mathbf{p}_i$	Melhor posição já encontrada pela partícula i
$\mathbf{g}$	Melhor posição já encontrada pelo enxame
w	Peso de inércia do PSO
$c_1,\ c_2$	Coeficientes de aceleração do PSO
$r_1,\ r_2$	Números aleatórios entre 0 e 1

Sumário

1 INTRODUÇÃO . . . 23
1.1 Contextualização . . . 23
1.2 Justificativa . . . 23
1.3 Questões de pesquisa . . . 24
1.4 Objetivos . . . 24
1.4.1 Objetivo geral . . . 24
1.4.2 Objetivos específicos . . . 25
1.5 Organização do trabalho . . . 25

2 ARQUITETURA DE HARDWARE . . . 27
2.1 Unidade de processamento embarcada . . . 27
2.2 Chassi, motores e encoders . . . 28
2.3 Drivers de motor . . . 29
2.4 Unidade de medição inercial . . . 29
2.5 Sensor LiDAR . . . 30
2.6 Câmera de profundidade . . . 30
2.7 Alimentação . . . 30
2.8 Integração dos componentes . . . 31

3 ARQUITETURA DE SOFTWARE . . . 33
3.1 Visão geral . . . 33
3.2 Sistema operacional e acesso ao hardware . . . 33
3.3 Ferramentas de validação dos componentes . . . 34
3.4 ROS 2 . . . 34
3.5 Nós embarcados no Raspberry Pi . . . 35
3.5.1 Driver do LiDAR . . . 35
3.5.2 Nó da base . . . 35
3.5.3 Driver da IMU . . . 36
3.5.4 Transformações estáticas . . . 36
3.6 Nós executados na estação de base . . . 36
3.6.1 Fusão de sensores . . . 36
3.6.2 Mapeamento . . . 37
3.6.3 Navegação . . . 37
3.7 Integração do software . . . 37
3.8 Plataforma para comparação de algoritmos de controle . . . 38
3.8.1 Níveis de controle . . . 38

3.8.2 Modelo da planta 39
3.8.3 Algoritmos previstos 39
3.8.4 Metodologia de comparação 40

REFERÊNCIAS 41

APÊNDICES 43

APÊNDICE A – PRIMEIRO APÊNDICE 45

APÊNDICE B – SEGUNDO APÊNDICE 47

ANEXOS 49

ANEXO A – PRIMEIRO ANEXO 51

ANEXO B – SEGUNDO ANEXO 53

1 Introdução

1.1 Contextualização

Robôs móveis autônomos deixaram de ser exclusivos de laboratórios e hoje aparecem em armazéns, hospitais, inspeção industrial e residências. Para se deslocar sozinho em um ambiente, um robô precisa resolver três problemas ligados entre si (SIEGWART; NOURBAKHSH; SCARAMUZZA, 2011). Ele precisa perceber o ambiente e construir um mapa dele. Precisa saber onde está nesse mapa. E precisa planejar e executar o movimento até o destino. Os dois primeiros problemas são tratados em conjunto pelas técnicas de localização e mapeamento simultâneos (SLAM). O terceiro depende do planejamento de caminho e, na base de tudo, do controle dos motores, que transforma os comandos de velocidade em movimento real das rodas.

O ROS 2 (*Robot Operating System 2*) se tornou a base comum para integrar essas partes (MACENSKI et al., 2022). Ele oferece drivers para sensores comerciais, algoritmos prontos de SLAM e navegação e um modelo de comunicação que permite distribuir o processamento entre vários computadores. Com isso, um robô funcional pode ser montado a partir de componentes acessíveis, como computadores de placa única, sensores LiDAR de baixo custo e motores com encoder.

O controle de movimento continua sendo uma área com muitas alternativas. O controlador PID é o mais usado na prática, pela simplicidade e pela robustez (ÅSTRÖM; HÄGGLUND, 2006). O controle preditivo baseado em modelo (MPC) trata explicitamente as restrições do sistema, ao custo de mais processamento (CAMACHO; BORDONS, 2007). Já os algoritmos bioinspirados, como a otimização por enxame de partículas (PSO), permitem sintonizar controladores sem um modelo exato da planta (KENNEDY; EBERHART, 1995). Cada abordagem tem vantagens e limitações que dependem do sistema controlado e do hardware em que ela roda.

1.2 Justificativa

Grande parte das comparações entre algoritmos de controle é feita em simulação. A simulação é útil, mas não reproduz bem efeitos presentes em um robô real: zona morta e saturação dos motores, atrito, escorregamento das rodas, ruído nos sensores, atrasos de comunicação e o limite de processamento do computador embarcado. Esses efeitos podem mudar qual algoritmo tem o melhor desempenho na prática.

Comparar algoritmos em um robô real, por outro lado, exige uma plataforma em

que o controle seja acessível em todos os níveis. Muitas plataformas comerciais usam controladoras de motor que fazem o controle de velocidade dentro da própria placa, o que impede trocar o algoritmo desse nível. Além disso, para que a comparação seja justa, todos os algoritmos precisam ser avaliados no mesmo hardware, com os mesmos ensaios e as mesmas métricas. Para isso, o software deve permitir trocar o controlador sem mexer no restante do sistema.

Este trabalho propõe o ARENA, uma plataforma robótica móvel construída com componentes de custo acessível. Os drivers de motor não fazem controle próprio, então o algoritmo de controle fica inteiramente no software. Do ponto de vista da Engenharia de Software, o projeto exige uma arquitetura modular, com interfaces bem definidas entre os drivers, a odometria, a navegação e os controladores. Isso permite incluir novos algoritmos sem alterar o resto do sistema. A plataforma também pode servir de base para outros trabalhos e para o ensino de robótica e controle.

1.3 Questões de pesquisa

Este trabalho busca responder às seguintes questões:

1. É possível construir, com componentes de baixo custo e ROS 2, um robô 4WD capaz de mapear um ambiente e navegar de forma autônoma até um ponto escolhido no mapa?
2. Como estruturar o software do robô para que diferentes algoritmos de controle possam ser trocados e comparados nas mesmas condições?
3. Como o PID clássico, o MPC e os algoritmos bioinspirados se comparam, em um robô real, quanto ao erro de seguimento, ao esforço de controle e ao custo computacional?
4. Quais desses algoritmos podem ser executados no computador embarcado e quais precisam ser executados na estação de base?

1.4 Objetivos

1.4.1 Objetivo geral

Desenvolver o ARENA, uma plataforma robótica móvel de tração nas quatro rodas baseada em ROS 2, capaz de mapear um ambiente e navegar de forma autônoma, e usá-la para comparar experimentalmente algoritmos de controle (PID clássico, controle preditivo baseado em modelo e algoritmos bioinspirados) sob as mesmas condições de ensaio.

1.4.2 Objetivos específicos

1. Montar e integrar o hardware do robô: computador embarcado, motores com encoder, drivers de motor, IMU, LiDAR, câmera de profundidade e sistema de alimentação;
2. Caracterizar experimentalmente os motores e os encoders e obter um modelo da dinâmica de cada lado do robô;
3. Desenvolver o nó da base em ROS 2, com controle de velocidade das rodas por algoritmo intercambiável, cálculo da odometria e parada automática em caso de falha de comunicação;
4. Integrar os sensores ao ROS 2 e combinar a odometria das rodas com a IMU por um filtro de Kalman estendido;
5. Construir o mapa do ambiente com o `slam_toolbox` e realizar a navegação autônoma até um ponto do mapa com o Nav2;
6. Implementar os controladores PID clássico, MPC e bioinspirados nos níveis de velocidade das rodas e de seguimento de trajetória;
7. Definir um conjunto padronizado de ensaios e métricas e comparar os algoritmos quanto ao desempenho e ao custo computacional;
8. Documentar e disponibilizar o código-fonte e os procedimentos de montagem e de ensaio, para que a plataforma possa ser reproduzida e ampliada.

1.5 Organização do trabalho

Este trabalho está organizado da seguinte forma. O Capítulo 2 descreve a arquitetura de hardware do robô: os componentes escolhidos, o funcionamento de cada um e a forma como se integram. O Capítulo 3 descreve a arquitetura de software: as ferramentas de validação dos componentes, a organização do sistema em nós do ROS 2, o fluxo de dados entre o robô e a estação de base e a estrutura usada para comparar os algoritmos de controle, incluindo a metodologia dos ensaios.

2 Arquitetura de Hardware

Este capítulo descreve os componentes físicos do robô, os critérios que orientaram a escolha de cada um, o princípio de funcionamento de cada componente e a forma como eles se integram. O robô é uma plataforma de tração nas quatro rodas (4WD) que deve mapear um ambiente com um sensor LiDAR e uma câmera de profundidade e, em seguida, deslocar-se de forma autônoma até um ponto escolhido no mapa. Ele também serve de plataforma para comparar algoritmos de controle, como o PID clássico, o controle preditivo e técnicas bioinspiradas (Seção 3.8). Esses objetivos determinam as exigências sobre o hardware: o robô precisa medir o próprio deslocamento (odometria) e a velocidade de cada roda, perceber o ambiente ao redor e transmitir esses dados a um computador com capacidade para processá-los.

A Tabela 1 resume os componentes adotados. As seções seguintes detalham cada um deles.

Tabela 1 – Componentes de hardware do robô

Função	**Componente**
Processamento embarcado	Raspberry Pi 3 (1 GB de RAM)
Estrutura	Chassi 4WD de 20 cm × 21 cm
Atuação	4 motores JGB37-520 (12 V, 333 rpm)
Odometria	Encoders Hall em quadratura (11 PPR)
Acionamento	2 drivers Cytron MD10C R3
Orientação	IMU Bosch BNO055 (UART)
Percepção 2D	LiDAR Slamtec RPLIDAR C1
Percepção 3D	Câmera Orbbec Astra Pro
Comunicação	Adaptador Wi-Fi USB (RTL8188EU)
Energia	Bateria LiPo 4S com BMS e dois conversores *buck*

Fonte: Autor.

2.1 Unidade de processamento embarcada

A unidade de processamento embarcada é um Raspberry Pi 3, um computador de placa única com processador ARM Cortex-A53 de quatro núcleos e 1 GB de memória RAM (RASPBERRY PI LTD, 2026). Ele foi escolhido por rodar um sistema Linux completo, o que permite usar o ROS 2 e as bibliotecas dos sensores sem precisar escrever drivers do zero. Ele também tem uma barra de 40 pinos de propósito geral (GPIO), com dois canais de PWM por hardware, uma interface UART e portas USB. Essas interfaces bastam para ligar todos os componentes do robô sem placas adicionais.

O Raspberry Pi 3 tem pouca capacidade de processamento e de memória para executar ao mesmo tempo o mapeamento simultâneo (SLAM), a navegação e a visualização. Por isso, ele foi usado apenas como computador de interface com o hardware. Ele lê os sensores, aciona os motores e envia os dados por Wi-Fi a um notebook, onde roda o processamento pesado. Essa divisão está descrita no Capítulo 3. Para liberar memória, a placa opera sem interface gráfica e é acessada remotamente por SSH.

2.2 Chassi, motores e encoders

A plataforma mecânica é um chassi 4WD de 20 cm de largura por 21 cm de comprimento, com rodas de 8 cm de diâmetro. O robô é do tipo *skid-steer*: as rodas não giram em torno de um eixo vertical, e as curvas são feitas girando os dois lados com velocidades diferentes. Essa configuração foi adotada por ser mecanicamente simples e por ter boa tração em pisos irregulares. Em troca, as rodas escorregam lateralmente durante as curvas, e isso afeta a odometria, como discutido na Seção 3.7.

Cada roda é movida por um motorredutor JGB37-520 de 12 V com velocidade nominal de 333 rpm no eixo de saída. A escolha considerou a tensão, compatível com a bateria por meio de um conversor, o torque oferecido pela caixa de redução e, principalmente, o encoder já integrado ao motor. Com rodas de 8 cm, a velocidade linear teórica sem carga é

$$v_{max} = \pi D \frac{n}{60} = \pi \cdot 0{,}08 \cdot \frac{333}{60} \approx 1{,}4 \text{ m/s}, \tag{2.1}$$

em que D é o diâmetro da roda em metros e n é a velocidade do eixo em rpm. Esse valor está bem acima do necessário para mapear ambientes internos, o que deixa margem para operar em velocidades baixas, que favorecem a qualidade do mapa.

O encoder de cada motor tem dois sensores de efeito Hall dispostos em torno de um ímã preso ao eixo do motor, antes da redução. Quando o ímã gira, cada sensor gera uma onda quadrada com 11 pulsos por volta do eixo do motor (PPR). Os dois sinais, chamados A e B, ficam defasados de 90°. Esse arranjo é chamado de quadratura. A ordem em que os sinais mudam de nível indica o sentido de giro, e contar as quatro bordas de cada ciclo (decodificação $\times 4$) multiplica a resolução por quatro. O número de contagens por volta da roda é então

$$N_{CPR} = 4 \cdot PPR \cdot R, \tag{2.2}$$

em que R é a relação de redução da caixa. Com esse valor, a distância percorrida pela roda em um intervalo com ΔN contagens é

$$\Delta s = \pi D \frac{\Delta N}{N_{CPR}}. \tag{2.3}$$

O valor de N_{CPR} é medido experimentalmente girando a roda um número conhecido de voltas, conforme descrito na Seção 3.3.

Os dois motores de cada lado recebem o mesmo comando, então medir uma roda por lado basta para estimar a velocidade daquele lado. Por isso, só os encoders dos motores dianteiros estão ligados ao Raspberry Pi. Os traseiros recebem apenas alimentação. Assim são usadas quatro entradas digitais em vez de oito, e a carga de interrupções no processador também cai pela metade.

2.3 Drivers de motor

Os pinos do Raspberry Pi fornecem poucos miliampères a 3,3 V. Por isso, os motores são acionados por dois drivers Cytron MD10C R3, um para cada lado do robô, com os dois motores daquele lado ligados em paralelo. O MD10C é uma ponte H, um circuito com quatro chaves eletrônicas (MOSFETs) que liga os terminais do motor à alimentação em uma polaridade ou na outra (CYTRON TECHNOLOGIES, 2026). Ele foi escolhido por operar na faixa de tensão do robô, por suportar com folga a corrente de dois motores em paralelo e por ser controlado com apenas dois sinais lógicos compatíveis com 3,3 V.

O controle segue o esquema sinal-magnitude. O pino DIR define o sentido de giro, e o pino PWM recebe uma onda quadrada cujo ciclo de trabalho (*duty cycle*) define a fração do tempo em que a ponte fica ligada. Como o motor responde à média da tensão aplicada, o ciclo de trabalho controla a velocidade. O PWM é gerado pelos periféricos de PWM por hardware do Raspberry Pi nos pinos GPIO12 e GPIO13, a 10 kHz. A geração por hardware não depende do agendamento do sistema operacional, então a frequência fica estável mesmo com o processador ocupado. Nos pinos de PWM há resistores de 10 kΩ para o terra, que mantêm os motores parados durante a inicialização do sistema, enquanto os pinos ainda não foram configurados.

2.4 Unidade de medição inercial

A odometria por encoders é pouco confiável para estimar a orientação de um robô *skid-steer*, porque as rodas escorregam nas curvas. Para medir a rotação de forma independente, foi usada a unidade de medição inercial (IMU) Bosch BNO055. Ela reúne acelerômetro, giroscópio e magnetômetro de três eixos e tem um microcontrolador interno que executa a fusão dos sensores e fornece a orientação já calculada, na forma de quatérnios ou ângulos de Euler (BOSCH SENSORTEC, 2026). Isso tira do Raspberry Pi o trabalho de calibração e de fusão e é o principal motivo da escolha.

O BNO055 aceita I^2C ou UART, e a interface é escolhida pelos pinos PS0 e PS1. Foi adotada a UART (PS0 em nível baixo e PS1 em nível alto), ligada aos pinos GPIO14 e GPIO15 com TX e RX cruzados. O sensor usa *clock stretching* no barramento I^2C,

recurso que o controlador I^2C do Raspberry Pi não trata de forma confiável. A UART evita esse problema. O sensor é alimentado pelo pino de 3,3 V da própria placa.

2.5 Sensor LiDAR

O sensor principal de mapeamento é o LiDAR Slamtec RPLIDAR C1. Ele gira um feixe laser em 360° e mede a distância até o obstáculo mais próximo em cada direção pelo tempo de voo da luz (SLAMTEC, 2026). No robô, o driver identificou o sensor operando em modo padrão com 5 mil medidas por segundo e 10 voltas por segundo, ou seja, cerca de 500 pontos por volta. Cada volta forma uma varredura 2D do plano em que o sensor está, a 10 cm do chão, e essa varredura é a entrada do algoritmo de SLAM.

O C1 foi escolhido por ter custo baixo, ter driver oficial para ROS 2 (`sllidar_ros2`) e alcance compatível com ambientes internos. Ele se comunica por UART a 460800 baud por meio de um adaptador USB-serial CP210x, ligado a uma porta USB do Raspberry Pi. O sensor foi instalado aproximadamente no centro do chassi, de modo que a posição medida fique próxima do centro de rotação do robô.

2.6 Câmera de profundidade

Para perceber obstáculos fora do plano do LiDAR, como mesas, degraus e objetos baixos, o robô tem uma câmera de profundidade Orbbec Astra Pro. Ela projeta um padrão de luz infravermelha estruturada sobre a cena e calcula a distância de cada pixel pela deformação desse padrão, observada por uma câmera infravermelha (ORBBEC, 2026). A câmera também tem um sensor RGB convencional. No robô, as duas partes aparecem como dispositivos USB distintos: o RGB pela classe UVC e a profundidade pelo SDK OpenNI2 da fabricante.

Nos testes, a profundidade foi lida a cerca de 30 quadros por segundo. Abaixo de aproximadamente 0,6 m a câmera não retorna medida, e esse é o seu alcance mínimo. A câmera foi escolhida pela disponibilidade e por ter drivers para ROS 2. O uso dela na navegação está previsto para uma etapa posterior, com resolução reduzida (320×240 a 15 quadros por segundo) para caber na capacidade do Raspberry Pi 3 e na banda da rede.

2.7 Alimentação

A energia vem de uma bateria LiPo de quatro células em série (4S), com tensão de 16,8 V quando carregada e circuito de proteção (BMS). Essa química foi escolhida pela relação entre energia e massa. Uma bateria de chumbo-ácido de 12 V e 7 Ah também

foi avaliada e descartada: ela pesa cerca de 2,1 kg, contra 0,3 kg a 0,5 kg da LiPo, e sua tensão de 11,5 V a 12,8 V é baixa demais para alimentar o conversor de 12 V.

A tensão da bateria é distribuída por dois conversores CC-CC abaixadores (*buck*), cada um com uma chave própria:

- um conversor de 12 V alimenta os dois MD10C (chave SW1). Como a tensão nos drivers fica regulada na tensão nominal dos motores, o ciclo de trabalho pode chegar a 100% sem sobretensão, e a velocidade para um mesmo ciclo de trabalho deixa de depender da carga da bateria;
- um conversor de 5 V e 5 A alimenta o Raspberry Pi (chave SW2, com voltímetro), que por sua vez alimenta pelas portas USB o LiDAR, a câmera e o adaptador Wi-Fi.

Separar a alimentação dos motores da alimentação da lógica evita que os picos de corrente na partida dos motores derrubem a tensão do computador. Mesmo assim, durante os testes o Raspberry Pi indicou subtensão (`vcgencmd get_throttled` retornou `0x70005`). Isso provocou redução de frequência do processador e reconexões dos dispositivos USB. A causa provável é a queda de tensão no cabo micro-USB e no fusível de entrada da placa quando o LiDAR, a câmera e o Wi-Fi funcionam juntos. A correção prevista é alimentar a placa diretamente pelos pinos de 5 V da barra de GPIO, com fio curto e de bitola maior, e, se necessário, trocar o conversor por um ajustável regulado entre 5,15 V e 5,2 V.

2.8 Integração dos componentes

A Tabela 2 mostra como os componentes se ligam ao Raspberry Pi. Eles usam três tipos de interface:

- **GPIO**: os sinais PWM e DIR dos drivers e os canais A e B dos encoders. Os encoders são alimentados com 3,3 V, então seus sinais já chegam no nível lógico do Raspberry Pi, sem conversor de nível;
- **UART**: a IMU BNO055;
- **USB**: o LiDAR, pelo adaptador CP210x, a câmera Astra Pro e o adaptador Wi-Fi.

Todos os circuitos compartilham o mesmo terra, que é a referência comum para os sinais lógicos entre o Raspberry Pi, os drivers e os sensores. Os pinos de terra foram distribuídos pela barra apenas para deixar cada fio de terra próximo do seu sinal. Os fios de potência dos motores ficam separados dos fios dos encoders para reduzir o ruído induzido nos sinais de quadratura.

Tabela 2 – Ligações no Raspberry Pi 3 (numeração BCM e pino físico)

Função	**GPIO**	**Pino**
MD10C esquerdo – PWM	12	32
MD10C esquerdo – DIR	17	11
MD10C direito – PWM	13	33
MD10C direito – DIR	22	15
Encoder dianteiro esquerdo – A / B	23 / 24	16 / 18
Encoder dianteiro direito – A / B	25 / 5	22 / 29
BNO055 – TX / RX	14 / 15	8 / 10
Buzzer	18	12
Alimentação dos encoders e da IMU	3,3 V	1 / 17

Fonte: Autor.

Na escolha dos pinos, dois pontos foram levados em conta. Primeiro, o GPIO18, usado pelo buzzer, está ligado internamente ao mesmo canal de PWM (PWM0) que o GPIO12. Por isso o buzzer é acionado apenas em liga/desliga ou por PWM por software, por meio de um transistor BC548 com resistor de base de 1 kΩ. Segundo, a saída de áudio analógico do Raspberry Pi usa os mesmos periféricos de PWM e foi desabilitada para não interferir nos motores. O esquemático completo foi desenhado no Proteus (revisão 1.1).

3 Arquitetura de Software

Este capítulo descreve o software do robô: a base do sistema operacional e das bibliotecas de acesso ao hardware, as ferramentas de teste desenvolvidas, a organização em nós do ROS 2, a forma como os dados fluem entre os sensores, os algoritmos de mapeamento e navegação e os motores, e a estrutura usada para comparar algoritmos de controle.

3.1 Visão geral

O software está dividido entre dois computadores que se comunicam por Wi-Fi. O critério da divisão é a capacidade de processamento. O Raspberry Pi 3 embarcado executa apenas o que precisa estar perto do hardware: os drivers dos sensores e o controle dos motores. O notebook, que funciona como estação de base, executa a fusão de sensores, o mapeamento, a navegação e a visualização, que exigem mais processador e memória. As duas partes usam a mesma distribuição do ROS 2, a Jazzy, o que garante que as mensagens trocadas sejam compatíveis.

O desenvolvimento seguiu duas fases. Na primeira, cada componente foi validado isoladamente com programas próprios em Python e C, sem ROS (Seção 3.3). Na segunda, os componentes validados passam a ser integrados como nós do ROS 2 (Seções 3.4 a 3.7).

3.2 Sistema operacional e acesso ao hardware

O Raspberry Pi executa o Raspberry Pi OS "Trixie" de 64 bits, com Python 3.13. A interface gráfica foi desativada (`multi-user.target`) para liberar memória, e o acesso é feito por SSH a partir do notebook.

O acesso aos pinos de GPIO é feito pela biblioteca pigpio (JOAN, 2026). Ela foi escolhida por três motivos. Ela dá acesso aos periféricos de PWM por hardware, necessários para acionar os MD10C com frequência estável. Ela amostra os pinos de entrada por DMA e entrega cada borda com o instante em que ocorreu, o que permite contar os pulsos dos encoders sem depender de interrupções do sistema operacional. E ela funciona como um serviço (`pigpiod`) que atende vários programas ao mesmo tempo. Uma consequência desse modelo é que o PWM continua ativo se o programa que o configurou terminar de forma anormal. Por isso, todos os programas que acionam motores zeram o PWM ao sair, e existe um comando de emergência para isso. Como o Trixie não traz o pacote da pigpio, ela foi compilada a partir do código-fonte.

3.3 Ferramentas de validação dos componentes

Antes da integração com o ROS 2, cada componente foi testado com um programa próprio. Isso permite separar defeitos de montagem de defeitos de software.

- **`teste_motores.py`**: bancada de teste dos motores e encoders, executada com as rodas suspensas. Ela tem um modo para cada etapa da caracterização. O modo `encoder` mede as contagens por volta da roda (Eq. 2.2) e confere o sentido de contagem. O modo `direcao` confere se cada lado gira no sentido comandado e se o encoder concorda. O modo `varredura` aumenta o ciclo de trabalho em passos de 5% para levantar a zona morta e a curva de velocidade em regime. O modo `degrau` aplica um degrau de tensão e amostra a velocidade a 100 Hz para estimar a constante de tempo dos motores. Todos os comandos passam por rampas, e o programa interrompe o teste se um motor recebe tensão alta e o encoder não registra movimento, o que indica roda travada ou encoder desligado;
- **`teste_encoders.py`**: mostra ao vivo os níveis lógicos, o número de bordas de cada canal e a posição de cada encoder, para diagnosticar fios soltos ou trocados;
- **`controle_wasd.py`**: teleoperação pelo teclado, via SSH. Por segurança, o robô para sozinho se as teclas deixam de chegar, se a conexão SSH cai ou se um motor trava;
- **`lidar.py`** e **`ver_lidar.py`**: leitura do RPLIDAR C1 no Raspberry Pi e exibição das varreduras em gráfico polar no notebook, com os dados enviados pela conexão SSH;
- **`profundidade.py`** e **`astra_depth.c`**: leitura da profundidade da Astra Pro e envio ao notebook como vídeo MJPEG colorido.

A leitura da câmera de profundidade exigiu uma solução específica. O pacote `openni` do repositório do Python não é compatível com o Python 3.12 ou mais recente. Depois de corrigido, ele ainda corrompia a memória, porque as estruturas de dados declaradas no pacote não correspondiam às da biblioteca da Orbbec. A solução foi escrever um programa em C compilado diretamente contra os cabeçalhos do SDK OpenNI2 da fabricante. Ele lê os quadros de profundidade e os repassa ao programa em Python. Na integração com o ROS 2, esse programa será substituído pelo driver da própria câmera e servirá apenas como ferramenta de teste.

3.4 ROS 2

O ROS 2 (*Robot Operating System 2*) é um conjunto de bibliotecas e ferramentas para software de robôs (MACENSKI et al., 2022). Um sistema em ROS 2 é formado por

nós, processos independentes que trocam **mensagens** de tipos padronizados por meio de **tópicos** com nome. Um nó que publica em um tópico não precisa saber quais nós o assinam. Por isso, um driver de sensor pode ser trocado sem mudar os algoritmos que consomem seus dados. A comunicação usa o padrão DDS, que descobre automaticamente os nós presentes na rede local. Assim, nós que rodam em computadores diferentes se comunicam como se estivessem no mesmo computador. É esse recurso que permite dividir o sistema entre o Raspberry Pi e o notebook.

O ROS 2 também mantém a árvore de transformações (TF), que registra a posição e a orientação relativas entre os sistemas de coordenadas do robô: o mapa, a origem da odometria, o centro do robô e cada sensor. Os algoritmos usam essa árvore para converter, por exemplo, um ponto medido pelo LiDAR para coordenadas do mapa.

O ROS 2 Jazzy tem suporte oficial ao Ubuntu 24.04, mas não ao Raspberry Pi OS Trixie nem ao Arch Linux, que são os sistemas usados no projeto. No Raspberry Pi, ele foi instalado a partir do repositório comunitário *rospian*, que distribui o Jazzy compilado para o Trixie arm64. Pacotes ausentes nesse repositório, como o driver do LiDAR, são compilados na própria placa com o `colcon`. No notebook, o ROS 2 roda em um contêiner Ubuntu 24.04 gerenciado pelo *distrobox*, que compartilha a rede e a interface gráfica com o sistema principal.

3.5 Nós embarcados no Raspberry Pi

3.5.1 Driver do LiDAR

O nó `sllidar_node`, do pacote oficial `sllidar_ros2` da Slamtec, lê o RPLIDAR C1 pela porta serial e publica cada volta no tópico `/scan`, como mensagem do tipo `LaserScan` (pacote `sensor_msgs`). Essa mensagem contém as distâncias medidas e os ângulos correspondentes.

3.5.2 Nó da base

Os drivers MD10C não têm controle de velocidade próprio. Eles só aplicam a tensão comandada. Por isso, o controle de velocidade das rodas roda no Raspberry Pi, em um nó próprio, chamado aqui de nó da base. Ele assina o tópico `/cmd_vel`, que contém a velocidade linear v e a velocidade angular ω desejadas para o robô, e calcula a velocidade de cada lado pelo modelo cinemático diferencial (SIEGWART; NOURBAKHSH; SCARAMUZZA, 2011):

$$v_e = v - \omega\frac{L}{2}, \qquad v_d = v + \omega\frac{L}{2}, \tag{3.1}$$

em que v_e e v_d são as velocidades das rodas esquerda e direita e L é a distância entre os centros das rodas dos dois lados.

Cada lado tem um controlador de velocidade que compara a velocidade desejada com a velocidade medida pelo encoder e calcula o ciclo de trabalho aplicado ao MD10C. O algoritmo desse controlador é intercambiável (Seção 3.8). O primeiro implementado é o PID:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \tag{3.2}$$

em que $e(t)$ é o erro de velocidade e K_p, K_i e K_d são os ganhos proporcional, integral e derivativo. Os parâmetros medidos na bancada (zona morta, curva de velocidade e constante de tempo) servem de ponto de partida para esses ganhos, para uma parcela de compensação direta (*feedforward*) e para o modelo usado pelos outros controladores.

No sentido contrário, o nó da base converte as contagens dos encoders em velocidades das rodas (Eq. 2.3) e calcula a velocidade do robô pela cinemática direta:

$$v = \frac{v_d + v_e}{2}, \qquad \omega = \frac{v_d - v_e}{L}. \tag{3.3}$$

Integrando essas velocidades ao longo do tempo, o nó estima a posição e a orientação do robô e as publica no tópico `/odom` (`nav_msgs/Odometry`). Por segurança, o nó para os motores se nenhum comando chegar em um intervalo curto, o que cobre falhas de rede e travamentos na estação de base.

3.5.3 Driver da IMU

O driver `bno055` (FLYNN, 2026) lê a IMU pela UART e publica a orientação, a velocidade angular e a aceleração no tópico `/imu` (`sensor_msgs/Imu`).

3.5.4 Transformações estáticas

A posição de cada sensor em relação ao centro do robô (`base_link`) é fixa e é publicada como transformação estática a partir de uma descrição do robô em URDF. Exemplos são a posição do LiDAR, próxima do centro do chassi e a 10 cm do chão, e a posição da IMU.

3.6 Nós executados na estação de base

3.6.1 Fusão de sensores

O pacote `robot_localization` implementa um filtro de Kalman estendido (EKF) que combina várias fontes de odometria em uma estimativa única do movimento (MOORE; STOUCH, 2016). Em um robô *skid-steer*, a rotação calculada pela Eq. (3.3) é imprecisa, porque as rodas escorregam lateralmente nas curvas. Por isso, o filtro usa dos encoders apenas a velocidade linear v e usa do giroscópio da IMU a velocidade angular

ω. Essa configuração é a mesma adotada no robô HelMoRo (HELBLING TECHNIK, 2026), que tem sensores semelhantes aos deste trabalho. O filtro publica a transformação `odom`→`base_link`. Por isso, o nó da base não deve publicá-la, para que a árvore de transformações não tenha duas fontes para o mesmo elo.

3.6.2 Mapeamento

O mapeamento usa o `slam_toolbox` (MACENSKI; JAMBRECIC, 2021). Esse pacote resolve o problema de localização e mapeamento simultâneos (SLAM): ele constrói o mapa a partir das varreduras do LiDAR e, ao mesmo tempo, estima a posição do robô nesse mapa. A estimativa da odometria serve de ponto de partida para alinhar cada nova varredura com as anteriores. Quando o robô volta a um lugar já visitado, o pacote reconhece o lugar e corrige o erro acumulado no caminho (fechamento de laço). O resultado é um mapa de ocupação 2D, uma grade com células livres, ocupadas ou desconhecidas, com resolução de 5 cm. O mapa pode ser salvo e reutilizado na navegação. O `slam_toolbox` publica a transformação `map`→`odom`, que corrige a deriva da odometria.

3.6.3 Navegação

A navegação autônoma usa o Nav2 (MACENSKI et al., 2020). A partir do mapa, da posição estimada e de mapas de custo que representam os obstáculos e uma margem de segurança em torno deles, o Nav2 planeja um caminho até o ponto de destino. Em seguida, um controlador local gera os comandos de velocidade em `/cmd_vel` que fazem o robô seguir esse caminho e desviar de obstáculos novos detectados pelo LiDAR. O destino é escolhido pelo usuário no RViz, a ferramenta de visualização do ROS 2.

Os comandos de velocidade podem vir do Nav2 ou da teleoperação. O pacote `twist_mux` escolhe entre essas fontes dando prioridade à teleoperação, para que o operador possa assumir o controle a qualquer momento.

3.7 Integração do software

A Tabela 3 resume os tópicos que ligam os nós. Os dados fluem em um ciclo fechado. O LiDAR, os encoders e a IMU publicam suas medidas no Raspberry Pi. O EKF, no notebook, combina a odometria das rodas com o giroscópio. O `slam_toolbox` usa essa estimativa e as varreduras do LiDAR para construir o mapa e localizar o robô. O Nav2 usa o mapa e a localização para gerar comandos de velocidade. Esses comandos voltam ao Raspberry Pi, onde o nó da base os converte em tensão nos motores.

A árvore de transformações segue a cadeia `map`→`odom`→`base_link`→{`laser`, `imu`}. Cada elo tem um único responsável: o `slam_toolbox`, o EKF e a descrição do robô, res-

Tabela 3 – Principais tópicos do sistema

Tópico	**Tipo**	**Publicado por**	**Usado por**
`/scan`	`LaserScan`	`sllidar_node` (Pi)	SLAM, Nav2
`/odom`	`Odometry`	nó da base (Pi)	EKF
`/imu`	`Imu`	`bno055` (Pi)	EKF
`/odometry/filtered`	`Odometry`	EKF (notebook)	Nav2
`/map`	`OccupancyGrid`	`slam_toolbox` (notebook)	Nav2, RViz
`/cmd_vel`	`Twist`	Nav2 ou teleoperação	nó da base (Pi)

Fonte: Autor.

pectivamente.

Como os nós estão em computadores diferentes, a rede faz parte do sistema. Só passam pelo Wi-Fi as varreduras do LiDAR, que são pequenas, as mensagens de odometria e IMU e os comandos de velocidade. A câmera de profundidade terá resolução e taxa de quadros reduzidas para não saturar o enlace. A descoberta automática do DDS usa *multicast*, que algumas redes institucionais bloqueiam. Nesse caso, a comunicação é feita por um roteador dedicado ou por um servidor de descoberta (*Discovery Server*) do próprio DDS.

3.8 Plataforma para comparação de algoritmos de controle

Além de mapear e navegar, o robô foi projetado como plataforma de testes para comparar diferentes algoritmos de controle no mesmo hardware e nas mesmas condições, como o PID clássico, o controle preditivo baseado em modelo (MPC) e técnicas bioinspiradas. Por isso, a arquitetura separa o algoritmo de controle do restante do sistema. Os drivers, a odometria, o mapeamento e a navegação são os mesmos em todos os ensaios, e só o controlador muda.

3.8.1 Níveis de controle

Os algoritmos podem ser comparados em dois níveis:

- **Controle de velocidade das rodas**, no nó da base, no Raspberry Pi. O controlador recebe a velocidade desejada de cada lado (Eq. 3.1) e a velocidade medida pelo encoder e calcula o ciclo de trabalho aplicado ao MD10C, limitado a ±100%. Todos os controladores desse nível seguem a mesma interface, e o algoritmo em uso é escolhido por um parâmetro do ROS 2, sem alterar o resto do nó;
- **Seguimento de trajetória**, no notebook. O controlador recebe o caminho planejado pelo Nav2 e a posição estimada do robô e gera os comandos de velocidade em

`/cmd_vel`. O Nav2 trata o controlador local como um módulo (*plugin*) que pode ser trocado por configuração. Assim, é possível usar os controladores que já vêm com ele ou escrever controladores próprios.

3.8.2 Modelo da planta

Vários dos algoritmos dependem de um modelo do sistema, seja para sintonizar ganhos, seja para prever o comportamento futuro. Esse modelo é obtido dos ensaios de bancada (Seção 3.3). A varredura de ciclo de trabalho fornece a zona morta e o ganho em regime K, e o ensaio de degrau fornece a constante de tempo τ. Com esses dados, cada lado do robô pode ser aproximado por um sistema de primeira ordem:

$$G(s) = \frac{\Omega(s)}{U(s)} = \frac{K}{\tau s + 1}, \tag{3.4}$$

em que $\Omega(s)$ é a velocidade da roda e $U(s)$ é o ciclo de trabalho, já descontada a zona morta. Para os controladores digitais, o modelo é discretizado no período de amostragem do nó da base.

3.8.3 Algoritmos previstos

PID clássico. É a referência de comparação, por ser o controlador mais usado na indústria (ÅSTRÖM; HÄGGLUND, 2006). Ele segue a Eq. (3.2), com os ganhos sintonizados a partir do modelo da Eq. (3.4) por métodos clássicos. Ele inclui proteção contra acúmulo do termo integral quando o ciclo de trabalho satura (*anti-windup*).

Controle preditivo baseado em modelo (MPC). A cada período de amostragem, o MPC usa o modelo da planta para prever a velocidade ao longo de um horizonte de N passos e escolhe a sequência de comandos que minimiza um custo, respeitando as restrições do sistema (CAMACHO; BORDONS, 2007):

$$\min_{u_0,\ldots,u_{N-1}} \sum_{k=1}^{N} (r_k - y_k)^2 + \lambda \sum_{k=0}^{N-1} \Delta u_k^2 \quad \text{sujeito a} \quad |u_k| \le 1, \tag{3.5}$$

em que r_k é a velocidade de referência, y_k é a velocidade da roda prevista pelo modelo, Δu_k é a variação do comando entre passos e λ pondera o esforço de controle. Só o primeiro comando da sequência é aplicado, e o cálculo se repete no período seguinte. A vantagem em relação ao PID é tratar diretamente a saturação do ciclo de trabalho. A desvantagem é o custo de calcular a otimização a cada período, que precisa caber no processador do Raspberry Pi 3. Por isso, horizonte, período de amostragem e formulação serão escolhidos considerando o tempo de cálculo medido na placa. No nível de seguimento de trajetória, executado no notebook, essa limitação é menor.

Algoritmos bioinspirados. São métodos de otimização inspirados em fenômenos naturais. Aqui eles são usados para sintonizar os parâmetros dos controladores diretamente

no robô físico, sem depender de um modelo exato. O primeiro método previsto é a otimização por enxame de partículas (PSO) (KENNEDY; EBERHART, 1995), aplicada aos ganhos do PID. No PSO, cada partícula representa um conjunto candidato de ganhos $\mathbf{x}_i = (K_p, K_i, K_d)$. A cada iteração, sua velocidade e sua posição são atualizadas por

$$\mathbf{v}_i \leftarrow w\,\mathbf{v}_i + c_1 r_1(\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2(\mathbf{g} - \mathbf{x}_i), \qquad \mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i, \tag{3.6}$$

em que $\mathbf{p}_i$ é a melhor posição já encontrada pela partícula, $\mathbf{g}$ é a melhor posição encontrada pelo enxame, w é o peso de inércia, c_1 e c_2 são coeficientes de aceleração e r_1 e r_2 são números aleatórios entre 0 e 1. Cada avaliação consiste em aplicar os parâmetros no robô, executar um ensaio e calcular um índice de desempenho. Cada tentativa deve durar pelo menos cerca de cinco constantes de tempo do motor, para que a roda chegue ao regime. A mesma estrutura serve para outros algoritmos bioinspirados e para sintonizar parâmetros de outros controladores, como os pesos do MPC.

Outros algoritmos. Como a interface do controlador é comum, novos algoritmos podem ser incluídos sem mudar o resto do sistema. Exemplos são o regulador linear quadrático (LQR), o controle *fuzzy* e o controle adaptativo.

3.8.4 Metodologia de comparação

Para que a comparação seja justa, todos os algoritmos são submetidos aos mesmos ensaios:

- degraus e rampas de velocidade em cada roda, com o robô suspenso e no chão;
- trajetórias de referência, como reta, curva e percurso fechado, executadas no mapa construído pelo `slam_toolbox`;
- repetição de cada ensaio para estimar a variabilidade dos resultados.

Os critérios de comparação são o erro de seguimento, medido por índices como a integral do erro absoluto (IAE), o sobressinal, o tempo de acomodação, o esforço de controle e o tempo de cálculo de cada algoritmo no processador em que ele roda. Os tópicos de referência, de medida e de comando são gravados com a ferramenta `ros2 bag` durante os ensaios, para que a análise seja feita depois com os mesmos dados.

Referências

ÅSTRÖM, K. J.; HÄGGLUND, T. *Advanced PID Control*. Research Triangle Park: ISA – The Instrumentation, Systems, and Automation Society, 2006. Citado 2 vezes nas páginas 23 e 39.

BOSCH SENSORTEC. *BNO055: Intelligent 9-axis absolute orientation sensor – Data sheet*. [S.l.], 2026. Disponível em: <https://www.bosch-sensortec.com/products/smart-sensor-systems/bno055/>. Acesso em: 28 out. 2026. Citado na página 29.

CAMACHO, E. F.; BORDONS, C. *Model Predictive Control*. 2. ed. London: Springer, 2007. Citado 2 vezes nas páginas 23 e 39.

CYTRON TECHNOLOGIES. *MD10C R3: User's Manual*. [S.l.], 2026. Disponível em: <https://www.cytron.io/p-10amp-5v-30v-dc-motor-driver>. Acesso em: 28 out. 2026. Citado na página 29.

FLYNN, E. *bno055: ROS 2 driver for the Bosch BNO055 IMU*. [S.l.], 2026. Disponível em: <https://github.com/flynneva/bno055>. Acesso em: 28 out. 2026. Citado na página 36.

HELBLING TECHNIK. *HelMoRo: Helbling Mobile Robot*. [S.l.], 2026. Disponível em: <https://helbling-technik.github.io/HelMoRo/>. Acesso em: 28 out. 2026. Citado na página 37.

JOAN. *pigpio library*. [S.l.], 2026. Disponível em: <https://abyz.me.uk/rpi/pigpio/>. Acesso em: 28 out. 2026. Citado na página 33.

KENNEDY, J.; EBERHART, R. Particle swarm optimization. In: *Proceedings of ICNN'95 – International Conference on Neural Networks*. [S.l.: s.n.], 1995. v. 4, p. 1942–1948. Citado 2 vezes nas páginas 23 e 40.

MACENSKI, S. et al. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, v. 7, n. 66, 2022. Eabm6074. Citado 2 vezes nas páginas 23 e 34.

MACENSKI, S.; JAMBRECIC, I. SLAM toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, v. 6, n. 61, p. 2783, 2021. Citado na página 37.

MACENSKI, S. et al. The marathon 2: A navigation system. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. [S.l.: s.n.], 2020. Citado na página 37.

MOORE, T.; STOUCH, D. A generalized extended Kalman filter implementation for the Robot Operating System. In: *Intelligent Autonomous Systems 13*. [S.l.]: Springer, 2016. p. 335–348. Citado na página 36.

ORBBEC. *Astra Series: especificações*. [S.l.], 2026. Disponível em: <https://www.orbbec.com/>. Acesso em: 28 out. 2026. Citado na página 30.

RASPBERRY PI LTD. *Raspberry Pi 3 Model B: documentação*. [S.l.], 2026. Disponível em: <https://www.raspberrypi.com/documentation/>. Acesso em: 28 out. 2026. Citado na página 27.

SIEGWART, R.; NOURBAKHSH, I. R.; SCARAMUZZA, D. *Introduction to Autonomous Mobile Robots*. 2. ed. Cambridge: MIT Press, 2011. Citado 2 vezes nas páginas 23 e 35.

SLAMTEC. *RPLIDAR C1: datasheet*. [S.l.], 2026. Disponível em: <https://www.slamtec.com/en/C1>. Acesso em: 28 out. 2026. Citado na página 30.

Apêndices

APÊNDICE A – Primeiro Apêndice

Texto do primeiro apêndice.

APÊNDICE B – Segundo Apêndice

Texto do segundo apêndice.

Anexos

ANEXO A – Primeiro Anexo

Texto do primeiro anexo.

ANEXO B – Segundo Anexo

Texto do segundo anexo.